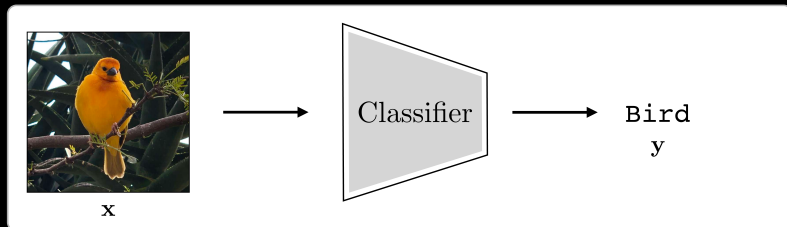
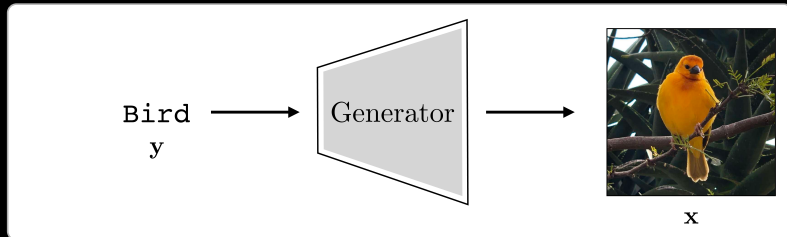


CLASSIFIER ONE WAY, GENERATOR THE OTHER



$f : \mathcal{X} \rightarrow \mathcal{Y}$, many to one

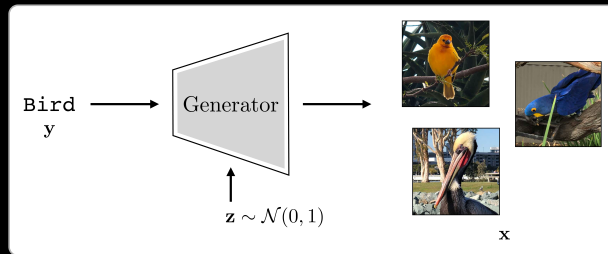


$g : \mathcal{Y} \rightarrow \mathcal{X}$, one to many

Many images all deserve the label “bird”, so f collapses them onto one answer. Run the arrow backwards and the ambiguity reappears: many images satisfy that same description, and **a deterministic network cannot be one to many**. Same input, same output, every time.

SO MAKE THE GENERATOR STOCHASTIC

The standard fix: keep the network deterministic, and feed it a **random input**. Now one label can produce many images.



$$z \sim p(Z), \quad x = g(z, y)$$

What z is

A vector of **latent variables**, drawn from something simple such as $\mathcal{N}(\mathbf{0}, \mathbf{I})$. It specifies everything about the image that the label does not: size, pose, position, background.

Why “latent”

Because it is never observed. The training set holds only images x and labels y ; **nobody wrote down** which z produced which scan.

Drop y and you get an **unconditional** model: data from scratch. That is the setting for the rest of this lecture.

WHY GENERATE MEDICAL IMAGES?

Augmentation

More training data, especially for the **rare cases** that the long tail never supplies. The fourth answer to Module 2's scarce-label problem.

Modality translation

CT $\leftrightarrow$ MRI. Generate the scan you did not acquire, so a model trained on one modality can be used on another.

Restoration

Denoising and super-resolution are generation *conditioned* on a degraded image: low-dose CT is the obvious case.

Sharing

Release *synthetic* images instead of patient data. Promising, and with a catch we return to at the end.

All four need the same thing: a model of $p(x)$ that we can **sample** from.

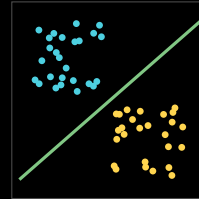
DISCRIMINATIVE VS. GENERATIVE

What we have done

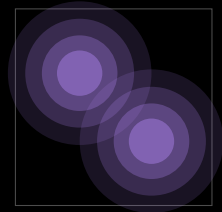
Learn $p(y \mid x)$: a **boundary**. Enough to label a scan, useless for producing one.

What we need now

Learn $p(x)$: the **data itself**, so we can draw $x \sim p(x)$ that was never in the training set.



$p(y \mid x)$



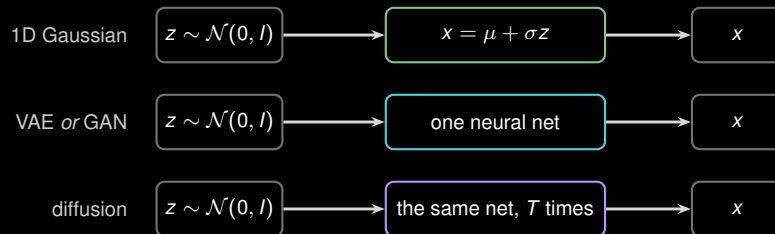
$p(x)$

A boundary separates. A density can be sampled.

The VAE: stable, but blurry

GENERATING = DRAW NOISE, APPLY A MAP

the *noise-to-data* map: this is the only thing that differs



Sampling is easy

All three draw the *same* noise. A VAE and a GAN generate **identically**: push z through one network. They are not different at sampling time at all.

Training is the hard part

Nobody hands you (z, x) pairs. Every method in this lecture is a different answer to: **how do you learn that map from data alone?**

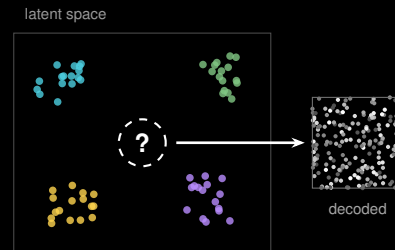
So the rest of the lecture is about *training*, not architecture.

START WITH AN AUTOENCODER, AND WATCH IT FAIL

- ▶ Compress x to a bottleneck z , reconstruct $\hat{x}$. Train with $\|x - \hat{x}\|^2$. It works.
- ▶ Now **sample a random z** and decode it.
- ▶ You get **noise**.

Why

Nothing ever asked the latent space to be *full*. Training only pinned down the points it saw; everything between them is undefined.



Sample between the clusters and the decoder has never been trained there.

VAE: ENCODE TO A *distribution*, NOT A POINT

What changed

The encoder no longer emits a code. It emits $\mu(X)$ and $\Sigma(X)$, the **parameters of a distribution**.

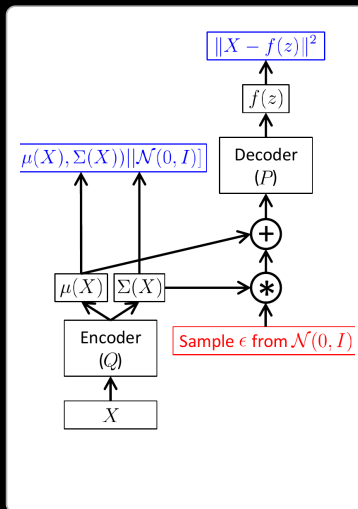
So each image claims a **region**, not a point. Push those regions towards $\mathcal{N}(0, I)$ and they overlap until they **tile the space**, so every z you might sample decodes to something plausible.



overlapping, no holes

Note what the encoder is for

It is **not used when you generate**. Sampling is the ladder we opened with: draw z , run the decoder. The encoder exists only to **train** the decoder, then it is discarded.



THE LOSS: RECONSTRUCTION + KL

To learn a noise-to-data map we would want (z, x) **pairs**, but we only have x . So learn **both directions at once**: data $\rightarrow$ noise with the encoder, noise $\rightarrow$ data with the decoder.

$$\mathcal{L} = \underbrace{\|x - \hat{x}\|^2}_{\text{reconstruction}} + \underbrace{\text{KL}(q(z | x) \parallel \mathcal{N}(0, I))}_{\text{regularise the latent}}$$

Drop the KL

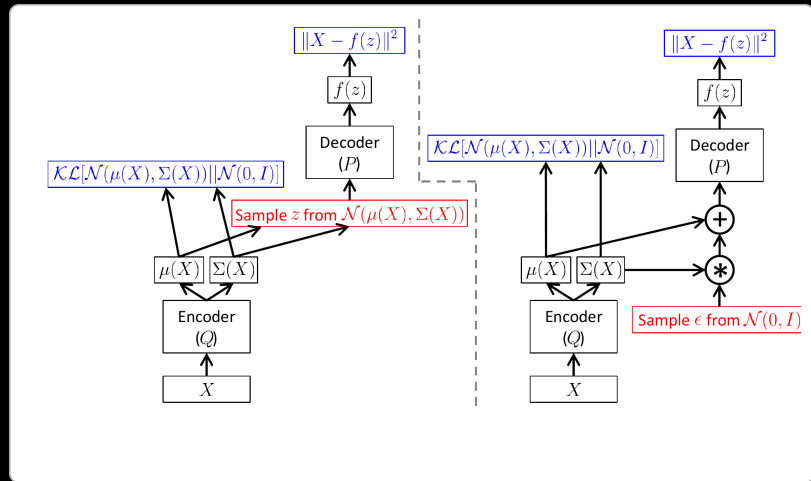
The regions shrink to points to reconstruct better, and you are back to the broken autoencoder we started the section with.

Drop the reconstruction

Every image maps to $\mathcal{N}(0, I)$, the latent carries no information, and the decoder emits the same blur for any z .

The **tension** between the two terms is the VAE. Weighting the KL higher (β -VAE) trades sharpness for a cleaner latent.

THE REPARAMETERISATION TRICK



Left: broken

"Sample z from $\mathcal{N}(\mu, \Sigma)$ " sits *on* the path from the loss back to the encoder. It is not differentiable, so backpropagation stops there.

Right: fixed

$z = \mu + \Sigma \odot \epsilon$ with $\epsilon \sim \mathcal{N}(0, I)$. Sampling moves **off** the path and becomes an input; what remains is $\times$ and $+$.

Both networks compute the same thing forward. Only the right one can be trained.